



# 講師用指導ガイド - タスク管理アプリワークショップ



## ワークショップ概要

### 対象者

- NITAC WEB班メンバー
- HTML/CSS/JavaScript初学者～中級者
- プログラミング経験: 0～2年程度

### 所要時間

- 全体: 3～4時間
- 導入・デモ: 15分
- Phase 1 (HTML): 45分
- Phase 2 (CSS): 90分
- Phase 3 (JavaScript): 60分
- まとめ・発表: 30分

### 事前準備

- ☐ 各レベルの教材の動作確認
- ☐ プロジェクター・画面共有の設定
- ☐ 参加者の開発環境確認 (VSCode、ブラウザ)
- ☐ 完成版デモの準備



## 学習目標の詳細説明

### 知識目標 (Knowledge)

- HTML:** セマンティックな要素の理解と適切な使用
- CSS:** Flexbox、アニメーション、レスポンシブデザインの実装
- JavaScript:** DOM操作、イベント処理、状態管理の基礎

### スキル目標 (Skills)

1. **問題解決**: エラーの特定と修正方法
2. **デバッグ**: ブラウザ開発者ツールの活用
3. **設計思考**: ユーザー体験を考慮したUI/UX設計

## 態度目標 (Attitude)

1. **自律学習**: 公式ドキュメントの参照習慣
2. **協働**: ペアプログラミングやコードレビュー
3. **創造性**: 基本機能を超えた独自アイデアの実装



## タイムスケジュール詳細

### 導入フェーズ (15分)

**0:00 - 0:05** アイスブレイク・自己紹介

- 参加者の経験レベル確認
- 今日の目標と期待の共有

**0:05 - 0:15** 完成版デモと全体像説明

- 完成版アプリの操作実演
- 各フェーズで学ぶ内容の概要
- 使用技術の簡単な説明

### Phase 1: HTML構造 (45分)

**0:15 - 0:30** HTML基礎レクチャー

トピック:

- セマンティックHTMLの重要性
- form要素とinput要素の使い方
- classとidの使い分け
- HTMLの構造化とアクセシビリティ

**0:30 - 0:55** ハンズオン実装

- 初心者: テンプレートガイド使用
- 中級者: 穴埋め問題解決
- 上級者: 最小ヒントで自力実装

**0:55 - 1:00** Phase 1振り返り

- 各自の実装確認
- 共通問題の解説
- 質疑応答

### Phase 2: CSS装飾 (90分)

**1:00 - 1:20 CSS基礎・レイアウトレクチャー**

トピック:

- CSSボックスモデル
- Flexboxレイアウト
- セレクタの詳細（クラス、疑似クラス）
- CSSカスタムプロパティ

**1:20 - 2:00 基本スタイリング実装**

- コンテナレイアウト
- ヘッダーとセクションデザイン
- フォーム要素のスタイリング

**2:00 - 2:10 休憩****2:10 - 2:25 アニメーション・エフェクトレクチャー**

トピック:

- CSS Transitions
- Transform プロパティ
- ホバーエフェクトの実装
- レスポンシブデザインの基礎

**2:25 - 2:25 高度なスタイリング実装**

- ホバーエフェクト追加
- アニメーション実装
- レスポンシブ対応

**2:25 - 2:30 Phase 2振り返り**

- スタイリング確認
- ブラウザ間差異の説明
- CSS最適化のヒント

## Phase 3: JavaScript機能（60分）

**2:30 - 2:45 JavaScript基礎レクチャー**

トピック:

- DOM（Document Object Model）の概念
- 要素の取得方法
- イベントリスナーの仕組み
- 関数の設計原則

**2:45 - 3:15 基本機能実装**

- 要素の取得
- イベントリスナーの設定
- チェックボックスの状態管理

**3:15 - 3:25 発展機能実装**

- 進捗バーの動的更新

- 完了メッセージの制御
- パフォーマンス最適化

### 3:25 - 3:30 Phase 3振り返り

- 機能動作確認
- JavaScriptデバッグ方法
- 拡張アイデアの共有

## まとめフェーズ（30分）

### 3:30 - 3:45 各自作品発表

- 実装した機能の紹介
- 苦労した点と学んだ点
- 独自に追加した機能

### 3:45 - 3:55 全体振り返り

- 学習目標の達成度確認
- 今後の学習指針
- 参考資料の紹介

### 3:55 - 4:00 クロージング

- フィードバック収集
- 次回活動の予告

## 各レベル別指導方針

### 初心者向け指導（🧑🏻）

#### 指導のポイント:

- 一つ一つの概念を丁寧に説明
- エラーメッセージの読み方を教える
- 成功体験を積み重ねる

#### フォロー方法:

1. 頻繁な進捗確認（5-10分間隔）
2. ペアプログラミングの推奨
3. 小さな成功を都度褒める
4. 「なぜそうなるのか」の理由説明

#### よくある質問と回答:

- Q: 「タグって何ですか？」 A: HTMLの要素を囲む印のようなもの。<div>のように書きます
- Q: 「クラスとIDの違いは？」  
A: クラスは複数使用、IDは一意。CSSでの指定方法も異なります
- Q: 「なぜ表示が崩れるんですか？」 A: タグの閉じ忘れが多い原因。一つずつ確認しましょう

## 中級者向け指導 (🎯)

### 指導のポイント:

- 「考えて解く」プロセスを重視
- 複数の解決方法を提示
- ベストプラクティスの紹介

### フォロー方法:

1. ヒントは段階的に提供
2. なぜその答えになるかを問いかける
3. 代替実装方法の討議
4. コードの可読性について議論

### 発展課題の提案:

- カスタムCSSアニメーションの追加
- JavaScriptでの入力バリデーション
- LocalStorageを使った状態保存

## 上級者向け指導 (🚀)

### 指導のポイント:

- 自主性を尊重
- 高度な技術的討議
- パフォーマンスとセキュリティの観点

### フォロー方法:

1. 設計思想について討議
2. コードレビューとリファクタリング提案
3. 最新の技術トレンドとの比較
4. チーム開発での実践方法

### チャレンジ課題:

- TypeScriptでの実装
- React/Vueでの再実装
- テスト駆動開発 (TDD) の実践
- パフォーマンス最適化



## トラブルシューティングガイド

### 技術的問題

#### HTML関連

**問題:** ページが真っ白になる

チェックポイント:

1. HTMLファイルのファイルパス
2. 基本的なHTML構造（<!DOCTYPE>, <html>, <head>, <body>）
3. タグの閉じ忘れ
4. ブラウザのコンソールエラー

**問題:** CSSが適用されない

チェックポイント:

1. <link>タグのhref属性
2. CSSファイルのファイルパス
3. class名の綴り間違い
4. CSSセレクタの書き方

## CSS関連

**問題:** レイアウトが崩れる

チェックポイント:

1. Flexboxのプロパティ（display: flex）
2. 親子関係の確認
3. widthとheightの設定
4. marginとpaddingの値

**問題:** アニメーションが動かない

チェックポイント:

1. transition プロパティの設定
2. ホバー時のプロパティ変更
3. ブラウザの対応状況
4. CSSの文法エラー

## JavaScript関連

**問題:** 要素が取得できない

チェックポイント:

1. DOMContentLoadedイベントの使用
2. セレクタの正確性（#id, .class）
3. 要素の存在確認（console.log）
4. HTMLとJavaScriptの実行タイミング

**問題:** イベントが発火しない

チェックポイント:

1. addEventListener の第二引数（関数参照）
2. 関数の定義と呼び出し
3. イベントタイプの正確性（'click', 'change'）
4. スコープの問題

## 環境・設定問題

### ブラウザ関連

問題: 古いブラウザでの表示問題

解決: モダンブラウザの使用推奨

- Chrome 80+
- Firefox 75+
- Safari 13+
- Edge 80+

## エディタ関連

問題: VSCodeの拡張機能

推奨拡張:

- Live Server
- HTML CSS Support
- Prettier
- Auto Rename Tag



# 評価・フィードバック

## 評価基準

### 知識理解（30%）

- ☐ HTML要素の適切な使用
- ☐ CSSプロパティの理解
- ☐ JavaScriptの基本文法

### 実装スキル（40%）

- ☐ 要求機能の実装完了度
- ☐ コードの可読性
- ☐ エラーハンドリング

### 問題解決能力（20%）

- ☐ エラーの自己解決
- ☐ デバッグツールの活用
- ☐ ヘルプリソースの利用

### 創造性・発展性（10%）

- ☐ 独自機能の追加
- ☐ デザインの改善
- ☐ ユーザビリティの考慮

## フィードバック方法

### リアルタイムフィードバック

実装中:

- 良い点をすぐに評価
- 改善点を具体的に指摘
- 次のステップを明確に示す

### 中間フィードバック（各フェーズ終了時）

フィードバック内容:

- 学習目標の達成度
- 技術的な成長点
- 次フェーズへの助言

### 最終フィードバック

総合評価:

- 全体的な成長の振り返り
- 強みと改善点
- 今後の学習指針
- 参考資料の提示



## 教材のカスタマイズ

### レベル調整

#### 易しくする場合

調整ポイント:

- より詳細なコメント追加
- 段階的なチェックポイント増加
- サンプルコードの提供
- ビジュアル説明の追加

#### 難しくする場合

調整ポイント:

- ヒントの削減
- 追加機能の要求
- パフォーマンス制約の設定
- コードレビューの実施

### 時間調整

#### 短縮する場合（2時間バージョン）

省略可能な内容:

- 詳細なCSS装飾
- JavaScript の発展機能
- 個別発表時間

## 延長する場合（6時間バージョン）

追加可能な内容:

- 詳細なレスポンス対応
- アクセシビリティ改善
- パフォーマンス最適化
- テスト実装



## 参考資料・追加学習

### 即座に参照できる資料

- [MDN Web Docs](#)
- [CSS-Tricks](#)
- [JavaScript.info](#)

### 継続学習のためのリソース

- [freeCodeCamp](#)
- [Codecademy](#)
- [Udemy Web Development Courses](#)

### 開発ツール

- [CodePen](#) - オンライン実行環境
- [JSFiddle](#) - コード共有
- [Can I Use](#) - ブラウザ対応確認



## 講師間の情報共有

### 事前準備会議

- ☐ 参加者のレベル確認
- ☐ 役割分担の決定
- ☐ 教材の最終確認
- ☐ 緊急時の対応策

### 当日の連携

- ☐ 開始前の環境確認
- ☐ 進行中の情報共有
- ☐ トラブル対応の分担

- ☐ 終了後の振り返り

## 事後フォロー

---

- ☐ 参加者からのフィードバック収集
- ☐ 教材の改善点整理
- ☐ 次回開催に向けた準備

---

このガイドを活用して、効果的で楽しいワークショップを実施しましょう！ 🎉